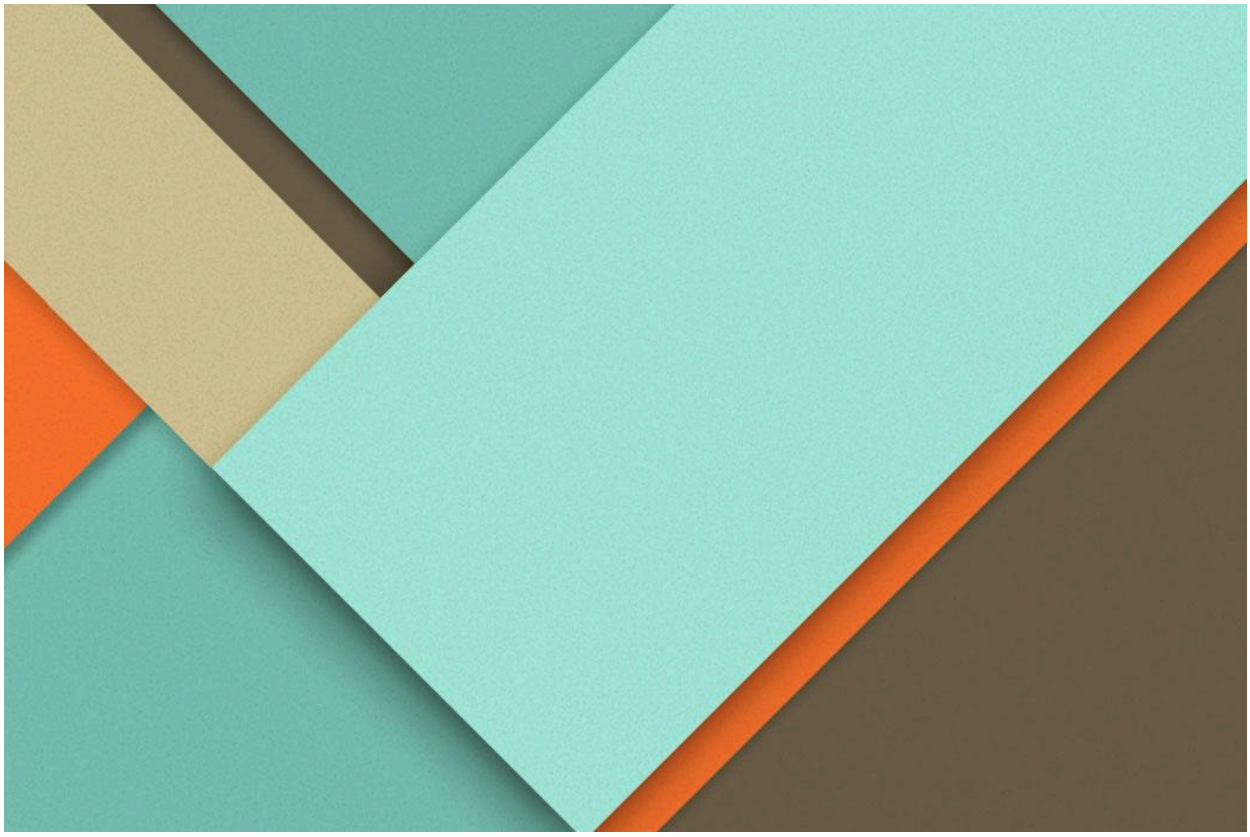


Plan, prepare and produce a functional business application.

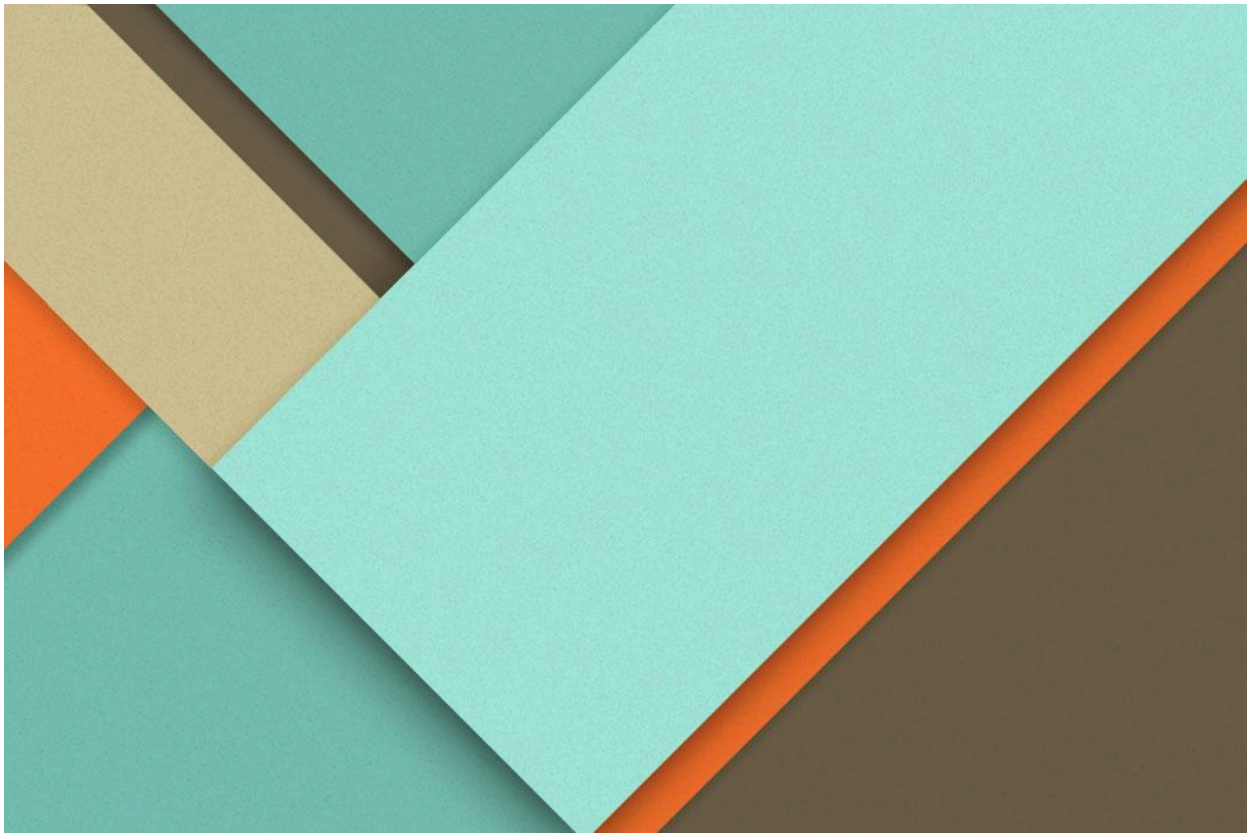
02.12.2024

Emily Bonnici

Table of Contents

Table of Contents 1
Purpose 4
User Requirements 5
I. Patients 5
II. Receptionists 5
III. Doctors 5
System Requirements 6
I. Functional Requirements 6
II. Non-functional Requirements 6
Areas of Risk in Developing the Proposed Application 7
I. Data Privacy and Security Risks 7
Mitigation Strategies: 7
II. Scalability Challenges 7
Mitigation Strategies: 7
III. System Downtime Risks 7
Mitigation Strategies: 7
IV. Usability Concerns 8
Mitigation Strategies: 8
V. Development Timeline & Deadlines 8
Mitigation Strategies: 8
VI. Change Management Risks 8
Mitigation Strategies: 8
Design and Development Tools 9
I. Design Tools 9
II. Development Tools 9
III. Database 9
Methodology and Design Patterns 10
I. Development Methodology 10
Key Features of the Waterfall Method: 10
Advantages of the Waterfall Method: 10
Disadvantages of the Waterfall Method: 11
Applicability to project: 11
II. Design Patterns 13
III. System Architecture 13

Development Process **14**
I. Design Phase 14
II. Testing Phase 14
Evaluation **15**
I. Development Tools 15
II. Methodology Review 15
III. Solution Impact 15
Link to DomainHospitalApplication **15**
Link to Video Demonstration **15**
Peer Review **17**
Use Case Diagram improvements: 18
UML Class Diagram improvements: 18
Evaluation Report **19**
1. Performance Against Problem Definition 19
2. Performance Against Initial Requirements 19
Patients Initial Requirements vs final performance: 19
Initial Functional and Non-functional Requirements vs final performance: 20
Functional Requirements 20
Functional Requirements 20
3. Critical Review of Development Stages 21
The design stage 21
The development stage 21
The testing stage 21
Conclusion **25**
References **26**
Appendices **27**



Part 1

Software Design Document

Purpose

The purpose of this software application is to comprehensively manage patients appointments for Domain Healthcare, a private hospital. The application will enable the Administration Office to efficiently handle scheduling and rescheduling of appointments, patient check-ins, including any walk-ins, daily treatment schedules, billing and record-keeping across the various hospital departments (Sommerville, 2016).

As the backbone of Domain Healthcare's operations, it is of utmost importance that the application functions correctly and accurately as any disruptions can lead to interrupted services, negatively impacting both patient care and revenue. Therefore, the application should be able to handle a large volume of changes real-time, without any delays, and intelligently assign and propose appointment dates according to the doctors availability across multiple departments (Sommerville, 2016).

User Requirements

I. Patients

A. Ability to book, reschedule and cancel appointments by calling or visiting the hospital

B. Notifications for upcoming appointments to avoid “no-show” penalties

II. Receptionists

A. Tools to schedule, update and cancel appointments real-time and efficiently
B. Daily treatment schedule report for doctors
C. Real-time view of available doctors and treatment rooms
D. Billing and payment system

III. Doctors

A. Access and update patient medical history and prescription records

System Requirements

I. Functional Requirements

A. Appointment management module with rescheduling and cancellation features

B. Daily report generation system for doctors

C. Integrated billing and payment management system

D. Prescription management system accessible by both doctors (doctors can see all their patients' records) and patients (who can only see their own records)

II. Non-functional Requirements

A. Scalability: System should support increasing of patients and doctors numbers

B. Reliability: System should ensure high availability with minimal downtime at all times

C. Usability: System should be intuitive for all users

D. Security: Systems should protect sensitive data with encryption and role-based access

E. Performance: System should handle concurrent users without difficulties

Areas of Risk in Developing the Proposed Application

To ensure the successful development of the application, the following risks have been identified and reviewed:

I. Data Privacy and Security Risks

The system will store and process sensitive patient data, including medical history, appointments, and billing information. Hence, unauthorised access or data breaches could result in legal issues and loss of trust (AWS, n.d.).

Mitigation Strategies:

- Implement encryption protocols for data storage and transfer.
- Use role-based access controls to limit data access to authorised personnel only.
- Regularly update the system to address any security vulnerabilities.

II. Scalability Challenges

The system may need to handle increasing numbers of patients, doctors, and appointments as the hospital grows.

Mitigation Strategies:

- Design a scalable system to accommodate growth.

III. System Downtime Risks

Downtime during could disrupt hospital operations, leading to missed appointments.

Mitigation Strategies:

- Ensure high availability with redundant systems and failover mechanisms.
- Monitor system performance in real-time and proactively address issues.

IV. Usability Concerns

If the system is not user-friendly, staff might face difficulties adopting it, leading to errors and inefficiencies.

Mitigation Strategies:

- Create user friendly processes.
- Provide intuitive interfaces with clear workflows and instructions.

V. Development Timeline & Deadlines

Development timeline and deadlines are limited, which may lead to uncompleted, untested processes.

Mitigation Strategies:

- Adopt agile development methodologies, to maintain flexibility and prioritise key features (Beck et al., 2001).
- MoSCoW strategy.

VI. Change Management Risks

Resistance to change from hospital staff could hinder the software's adoption and usage.

Mitigation Strategies:

- Communicate the benefits of the system early and consistently.
- Conduct peer review and evaluation.

Design and Development Tools

I. Design Tools

A. draw.io has been chosen for its simplicity, versatility and free features. It supports the creation of essential diagrams like entity-relationship diagrams (ERDs) and workflows, which are critical for visualizing database relationships and user processes. Its cloud integration enables real-time collaboration, while its free access and export options in multiple formats make it ideal for student projects. Using draw.io ensures the design phase is efficient, professional, and well-structured (Lucidchart, n.d.).

II. Development Tools

A. Apache NetBeans 23 was chosen as the Integrated Development Environment (IDE) for the development of this project as it provides support for Comprehensive Java Support with built-in tools, intuitive and User-Friendly Interface, great Built-In Features that do not require external plugins, intuitive GUI builder for Swing components, Scalability and Future Integration ideal for future enhancements to the project, and a very active community and extensive documentation.

III. Database

A. For this project, a traditional database will not be implemented. Instead, file serialization will be used to store and manage data. Serialization allows Java objects, such as appointments, patients, and staff, to be converted into a byte stream and saved to files, which can then be deserialized to restore the object states as needed. Although this approach is suitable for the current project's, a relational or NoSQL database would be a better future option.

Methodology and Design Patterns

I. Development Methodology

A. Agile Methodology:

Allows iterative and incremental development, enabling regular user feedback and adapting to changing requirements (Beck et al., 2001).

B. Waterfall Method:

The Waterfall method is a linear and sequential software development model where each phase is completed before moving on to the next. It is one of the oldest and most traditional approaches to software development, characterised by distinct phases such as Requirements Gathering, Design, Implementation, Testing, Deployment, and Maintenance (Pressman and Maxim, 2020). Unlike Agile, the Waterfall method emphasises planning and documentation over iterative feedback and adaptability.

Key Features of the Waterfall Method:

- **Sequential Workflow:** Each phase must be completed before the next begins, ensuring clarity and reducing overlaps.
- **Well-Defined Deliverables:** Each stage has clearly defined goals and outputs.
- **Extensive Documentation:** The model relies on documentation to ensure all requirements and designs are fully specified (Sommerville, 2016).
- **Low Flexibility:** Once a phase is completed, changes are difficult and expensive to implement.

Advantages of the Waterfall Method:

- **Simplicity:** The linear approach is easy to understand and implement, making it ideal for projects with stable requirements.
- **Clear Structure:** Each phase is well-defined, ensuring all stakeholders understand the project's progress.
- **Documentation:** Detailed documentation helps with knowledge transfer and long-term maintenance.

- **Predictability:** Timelines, costs, and deliverables are established early in the process (Pressman and Maxim, 2020).

Disadvantages of the Waterfall Method:

- **Limited Flexibility:** Adapting to changes or incorporating new requirements can be challenging and costly.
- **Risk of Misinterpretation:** Errors in the early phases (e.g., requirements gathering) may not surface until testing or deployment, potentially causing major rework.
- **Delayed Feedback:** Stakeholder feedback is often received late in the development cycle, increasing the risk of misaligned outputs.
- **Unsuitable for Complex or Dynamic Projects:** Projects requiring frequent changes or those with evolving requirements are better suited to iterative methods like Agile (Beck et al., 2001).

Applicability to project:

The waterfall method is suitable in scenarios where:

- The requirements are well-understood and unlikely to change significantly.
- A high level of documentation is required to meet regulatory or compliance standards.
- The project is small and has limited stakeholder involvement, reducing the need for iterative development.

Therefore, the waterfall method will be used for the development of the hospital management software.

C. MoSCoW Prioritisation:

The MoSCoW method categorises requirements based on their priority:

Must-Have

- Core appointment scheduling, modification, and cancellation.
- Daily treatment schedule generation for doctors.
- Billing calculation, issuance, and payment management.

Should-Have

- A patient portal for appointment view and accessing prescriptions.
- Support for walk-in patient registration.

Could-Have

- Integration with pharmacy systems for direct prescription fulfilment.
- Reporting features for financial analytics and operational statistics.

Won't-Have

- Telemedicine features such as video consultation.
- Live appointment access by Doctors.
- Scheduling, modification, and cancellation by patients.
- Scheduling, modification, and cancellation by Doctors.
- AI-driven recommendations for doctor assignments based on patient history.
- Notification system for upcoming appointments and bills.

II. Design Patterns

A. **Model-View-Controller (MVC):** Ensures separation of concerns for better maintainability and scalability (Sommerville, 2016).

B. **Observer Pattern:** Used for real-time updates, such as sending appointment reminders.

III. System Architecture

A. **Singleton Pattern:** The singleton pattern is a design pattern that ensures a class has only one instance while providing a global point of access to that instance. This is achieved by restricting the instantiation of the class and using a static method to control object creation (Gamma et al., 1994). Some advantages of singleton pattern include controlled access by limiting instantiation of classes, ensuring shared resources are not duplicated which can lead to conflicts and errors, a central access point making it easier for modules to interact with shared resources as needed, and ease of maintenance for simplified debugging and future enhancements are required. Hence the singleton pattern aligns with the hospital's goal of both scalability and reliability.

Development Process

I. Design Phase

A. **Use Case Diagrams:** Used to plan and show functional requirements of the system by mapping out the interactions between users and the system (*refer to Figure 1 under Appendices*).

B. **UML Class Diagram:** Used to define the static structure of the system by depicting the system's classes, their attributes, methods and relationships accordingly (*refer to Figures 2 and 3 under Appendices*).

C. **Wireframes:** Visual prototypes of the user interface (UI), outlining the placement of elements like buttons, menus and forms *refer to Figures 4 to 13 under Appendices*).

Development Phase

D. **Backend Development:** Use Java backend logic to ensure performance and scalability.

E. **Frontend Development:** Develop using java for a responsive and interactive user interface.

II. Testing Phase

A. **Unit Testing:** Validate individual system components.

B. **Integration Testing:** Ensure modules like scheduling work together seamlessly.

C. **User Acceptance Testing (UAT):** Conduct a peer review for feedback and self testing.

Evaluation

I. Development Tools

NetBeans was chosen as the Integrated Development Environment (IDE) for the development of this project as it provides support for Comprehensive Java Support with built-in tools, intuitive and User-Friendly Interface, great Built-In Features that do not require external plugins, intuitive GUI builder for Swing components, Scalability and Future Integration ideal for future enhancements to the project, and a very active community and extensive documentation.

II. Methodology Review

Agile's iterative approach ensures that stakeholder feedback is incorporated continuously, making it superior to Waterfall for dynamic requirements (Beck et al., 2001).

III. Solution Impact

By automating appointment scheduling, treatment schedules, and billing, the application addresses inefficiencies in manual processes. Its modular design ensures that the system can grow alongside the hospital's future operational needs.

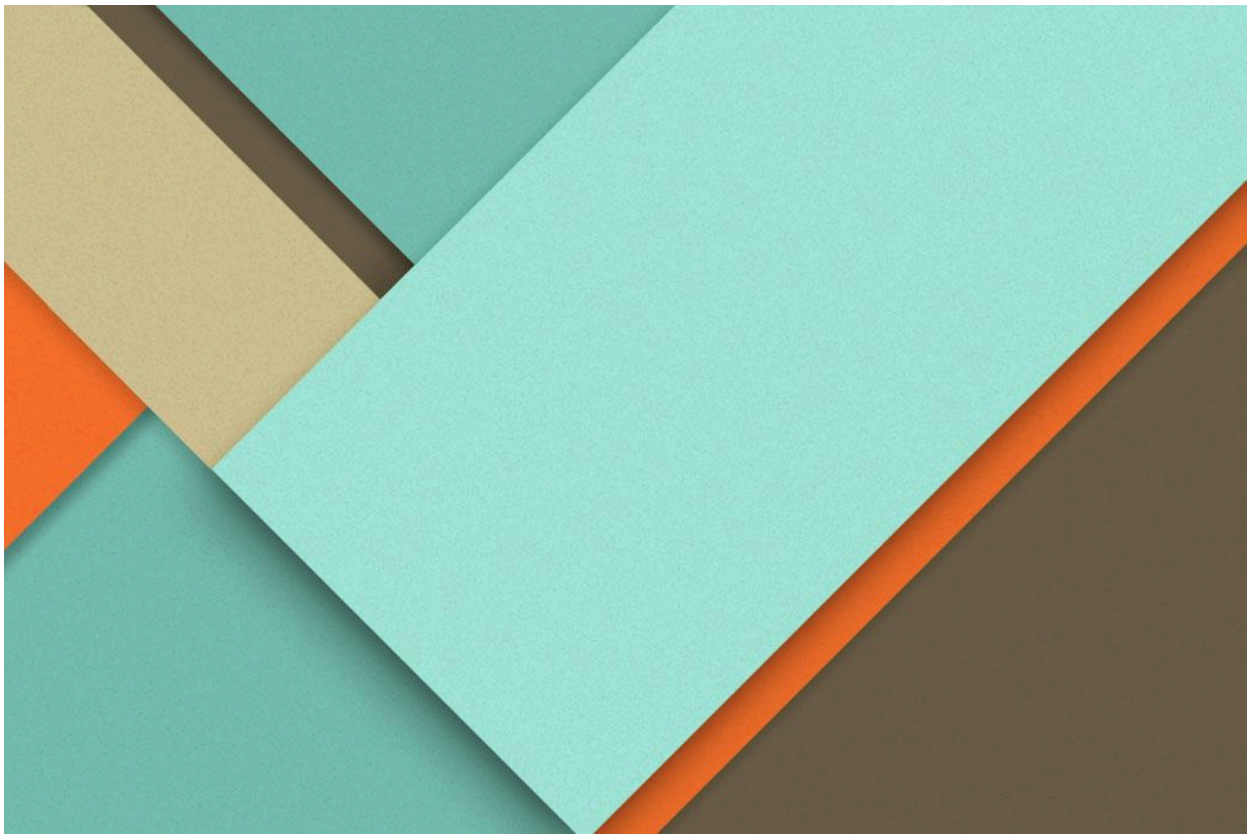
Link to DomainHospitalApplication

DomainHospitalApplication Project can be found here:

https://drive.google.com/drive/folders/1XZ3JuOHXHO59s9cSvlMH3aR46N1rEYX8?usp=sharing

Link to Video Demonstration

https://drive.google.com/drive/folders/1OvrsEyia3Gd_kBJAVenQEU7oL-y_yhyE?usp=sharing



Part 2

Peer Review and Evaluation Document

Peer Review

As part of the review process, the Software Design Document together with the working Software Application have been sent to Mr Kurt Ellul to conduct the following peer review:

- The problem definition statement
- Proposed solution
- Development strategy

Upon reviewing the above mentioned elements, Mr Kurt Ellul provided valuable feedback as illustrated below:

Peer Review of UML Diagrams and Design Patterns

1. UML Use Case Diagram

- **Areas for Improvement:**
  - The use case diagram does not fully capture the expected functionalities or correctly represent the relationships between actors and use cases.
  - It's important to include accurate relationships such as "include" or "extend" where appropriate and ensure all primary and secondary use cases are accounted for.
  - I recommend referring to the documentation I will share for guidance on structuring and detailing a use case diagram effectively.

---

2. UML Class Diagram

- **Strengths:**
  - The class diagram does show an attempt at capturing the system's architecture, which is commendable.
- **Areas for Improvement:**
  - The lack of visible aggregation and relationships among classes is a critical gap, as these links are crucial for understanding system interactions and dependencies.
  - Please review the examples and documents provided to understand how to represent class relationships, such as inheritance, associations, aggregations, and multiplicities.
  - Ensure that visibility modifiers (e.g., private, public) and clear attribute/method definitions are included to enhance diagram readability.

---

3. Wireframes

- **Strengths:**
  - The wireframes are of high fidelity and provide an excellent representation of the user interface.
  - The layout promotes a smooth and effective user experience, demonstrating a good understanding of user-centric design principles.
  - The data flow through the interface is logical and aligns with best practices, ensuring intuitive navigation and usability.

---

4. Application of Design Patterns

- **Strengths:**
  - The effort to apply design patterns such as the factory, singleton, and iterator patterns is evident and appreciated. These choices show an awareness of modularity, scalability, and efficiency.
- **Suggestions for Improvement:**
  - While the patterns are implemented, consider documenting their integration with the UML diagrams to ensure alignment across your work. This would reinforce the cohesiveness of the design.

---

5. Recommendations and Resources

- To address the gaps in the UML diagrams, I recommend revisiting the fundamental principles of use case and class diagrams. The documents I will provide will serve as a helpful reference.
- Pay special attention to the following:
  - Ensuring that the use case diagram clearly reflects user interactions with the system.
  - Representing all necessary relationships, such as associations and aggregations, in the class diagram.

Compliments and Final Remarks

- While there are areas that need attention, your work demonstrates strong potential, particularly in the design of wireframes, which are intuitive and user-focused.
- Addressing the gaps in the UML diagrams will significantly enhance the overall quality and coherence of your project.
- Keep pushing forward, and don't hesitate to leverage the resources I'm sharing to strengthen your understanding and execution.

You're on the right track, and with a bit more refinement, your project will reach the high standard you're aiming for!

Use Case Diagram improvements:

After receiving the feedback from Mr Ellul on the Use Case Diagram, the suggestions provided were taken into consideration and the necessary changes have been included. The feedback highlighted areas for improvement to accurately capture all functionalities, represent relationships between actors and use cases, and appropriately use relationships like "include" and "extend." Hence the updated Use Case Diagram now accurately represents the application's functionalities and their corresponding relationships with actors.

UML Class Diagram improvements:

While these gaps and shortcomings between the diagrams provided by myself and the diagrams provided by Mr Ellul have been duly noted, it is also noted that, despite Mr Ellul's comments re "*visibility modifiers (e.g., private, public) and clear attribute/method definitions are included to enhance diagram readability.*" the UML Class Diagram already adheres to UML conventions. In fact, public visibility is denoted by the "+" symbol, and private visibility by the "-" symbol.

With regards to feedback received re relationships among classes, this has been duly noted and, while the convention is to show these in the same UML Class Diagram, these have been included and presented in a second UML Class Diagram.

Hence, given that the visibility modifiers are already implemented and the associated code has already been successfully developed, together with the 2nd UML illustrating the relationships among classes, no changes to the main UML Class Diagram have been made at this time.

Evaluation Report

1. Performance Against Problem Definition

The primary purpose of this software application is to comprehensively manage patient appointments and hospital operations for Domain Healthcare, a private hospital. While the application successfully enables the Administration Office to handle scheduling, rescheduling, and patient check-ins (including walk-ins), as well as daily treatment schedules, billing, and record-keeping across departments, there are some notable shortcomings in its implementation, which will be discussed below.

2. Performance Against Initial Requirements

Patients Initial Requirements vs final performance:

The initial requirements for patients included:

- Ability to book, reschedule and cancel appointments by calling or visiting the hospital.
- Notifications for upcoming appointments to avoid “no-show” penalties

Patients have successfully been given a Login Menu where they can view their appointments, however, even though the initial requirements specifically mention that patients are to get notification for upcoming appointments, this has not been implemented. This was due to focusing on completing the core functionalities within the available time and resources.

Receptionists Initial Requirements vs final performance:

- Tools to schedule, update and cancel appointments real-time and efficiently
- Daily treatment schedule report for doctors
- Real-time view of available doctors and treatment rooms
- Billing and payment system

While the tools to schedule, update and cancel appointments have been successfully implemented, during testing it has been noted that these functions could be made more user friendly. An example in question is when it comes to updating appointments: in order to do so, the receptionist must know the appointment ID. This creates an extra step as, if the patient who is calling doesn't have their appointment ID handy, the receptionist needs to generate a report to find it. Hence, in hindsight, a better option would have been to give different options to search by: search by date, search by patient ID, etc.

The Daily treatment schedule report for Doctors originally included three reports, however, one of them (getAllDoctorAppointmentsByDoctorID) has been commented out due to errors, hence it has not been implemented.

Another requirement which was not included is the room allocations for treatments. While this feature was initially intended to be part of the project, it was unintentionally omitted due to an oversight during the development and implementation of the application. This highlights the importance of dedicating more time to checking that all requirements have been included in the diagrams.

Billing and payment system has been implemented. Some improvements could include warnings when a client overpays, allocating payment against an appointment, automated price list depending on doctor or any other chosen field.

Doctor Initial Requirements vs final performance:

- Access and update patient medical history and prescription records

Doctors have been successfully given access to patients' records. Doctors have also been given the option to see schedules, however no rights to export them have been assigned as these are to be done solely by the receptionists.

Initial Functional and Non-functional Requirements vs final performance:

Functional Requirements

- Appointment management module with rescheduling and cancellation features
- Daily report generation system
- Integrated billing and payment management system
- Prescription management system accessible by both doctors (doctors can see all their patients' records) and patients (who can only see their own records)

All the functional requirements above have been successfully implemented.

Functional Requirements

- Scalability: System should support increasing of patients and doctors numbers
- Reliability: System should ensure high availability with minimal downtime at all times
- Usability: System should be intuitive for all users
- Security: Systems should protect sensitive data with encryption and role-based access
- Performance: System should handle concurrent users without difficulties

All the above non-functional requirements have been successfully implemented.

3. Critical Review of Development Stages

The design stage

The design overall meets the intended purposes with different functionalities for patients, appointments, the remaining classes and separate UIs etc. However, with every project there are risks associated. One of the main risks that was encounter during the design phase was the risk of insufficient time allocated for the design phase or underestimation of required resources, which resulted in a rushed design that may have overlooked some critical details as mentioned above.

Another risk identified during the design phase was the decision to use file-based serialization for data storage instead of implementing a database. Serialization works well for basic saving and loading of data, but it can become less efficient as the amount of data grows. It also lacks the flexibility and robustness of a database, such as handling multiple users or performing complex queries. This choice might make it harder to scale the application or integrate new features in the future.

The development stage

The application makes good use of Object-Oriented Programming principles, with modular classes like Doctor, Patient, and Staff. It also consistently uses logical layers, such as ApplicationLogic, to separate responsibilities, which helps keep the application organised and easier to maintain. However, there are a few areas that could be improved. For example, there is some repeated validation logic in the UI modules, such as PatientUI and StaffUI, which could make the code harder to maintain in the future. Additionally, inheritance could have been used more effectively to manage shared attributes between entities like Doctor, Staff, and Patient, which would have made the code more reusable and easier to work with.

The testing stage

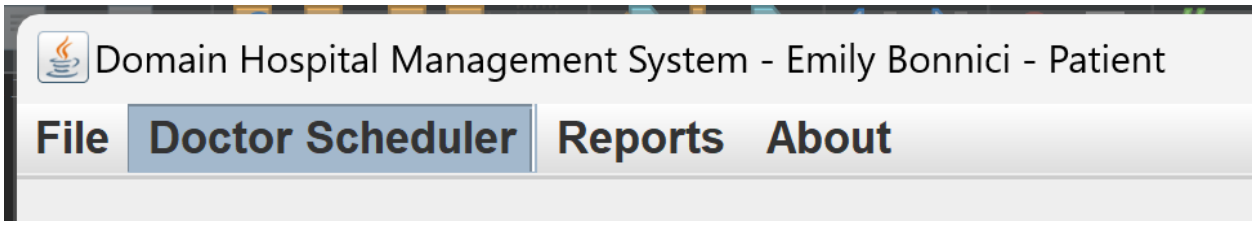
During the testing stage, the application showed some strengths, particularly in the validation and feedback mechanisms included in the UIs, which help reduce user errors during interactions. Testing coverage for important features like appointments and schedules also appears thorough. However, there are some areas where improvements could be made. For instance, edge case testing, such as handling simultaneous access to modify schedules, seems limited. Additionally, the lack of automated testing tools or

frameworks may make it harder to ensure consistent and scalable testing as the application grows.

Application could be improved by implementing better validation and data integrity measures. For example, integrating cross-validation between modules, such as ensuring payment records align with appointments or preventing overlapping schedules, would improve consistency. Additionally, using shared validators for input fields could further help reduce redundant code

A pharmacist role was originally planned to be implemented to handle the medications, however this was not expanded or worked on. Hence this too can be hidden entirely for a cleaner look.

It was also noted that all users, no matter the access rights, can view all menu bars. For example, a patient will still be able to see the full menu, but is limited to what he can click on:

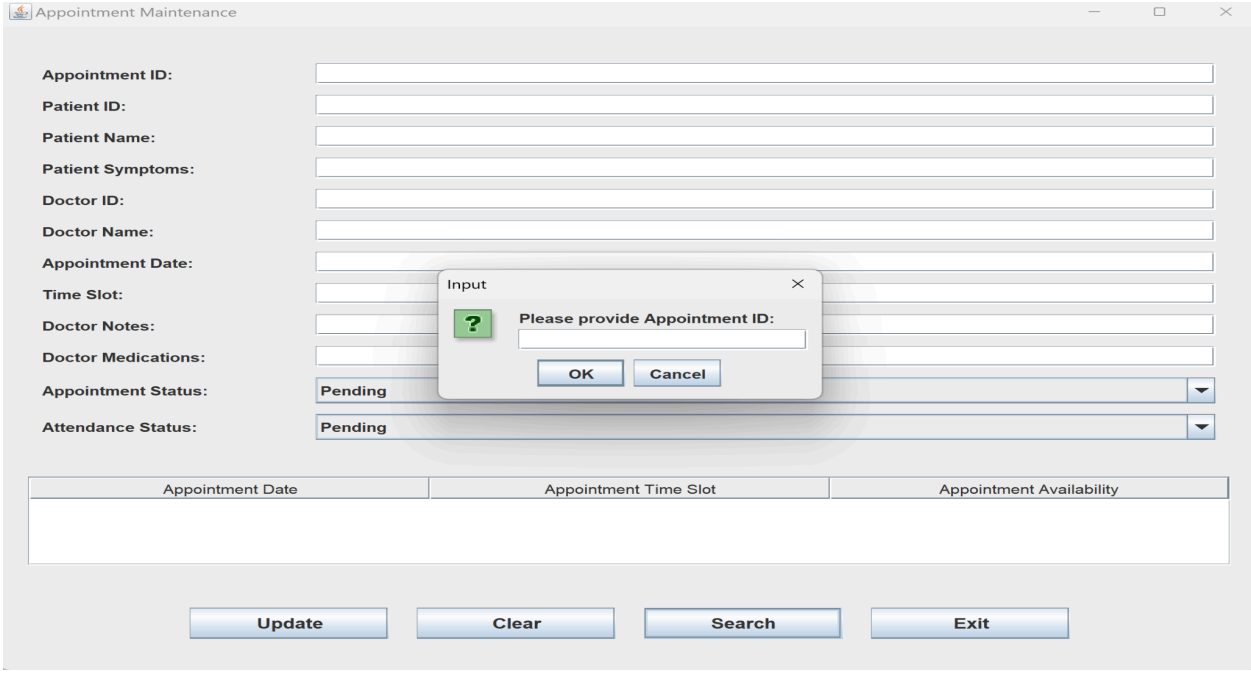


Given that patient cannot click on Doctor Scheduler and Reports, these can be hidden from the Patient view for a cleaner and more professional look.

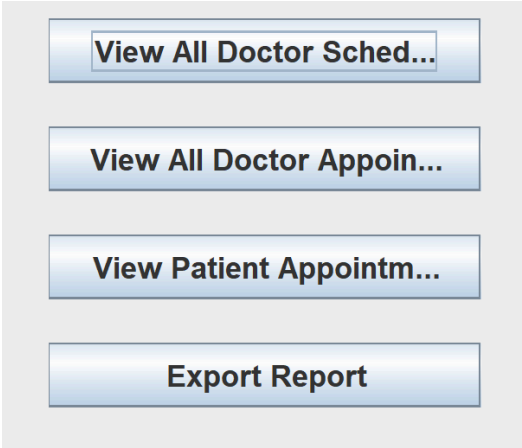
The payment menu does not properly handle partial payment. Hence it would be ideal if it allowed handling of partial payments, along with outstanding balance tracking. Another option that can be added is to include printable or exportable files for invoices generated.

Another improvement that can be made to the application is auto generation of age depending on DOB input, reducing the chances of incorrect data and also allowing the system to remain updated with the correct age throughout the years.

Another function that could be made clearer is the Appointment Maintenance page. It might not be clear that the user needs to click on 'search' before inputting the data:



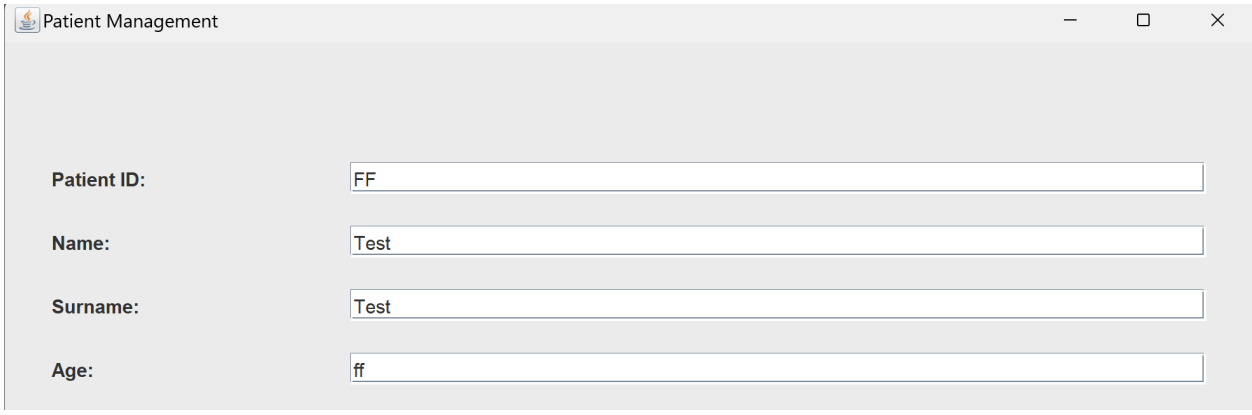
Something else that was noted is that the buttons below in the ReportUI did not fully display the text. This was a pretty easy fix, hence the button width has been increased for a cleaner look.



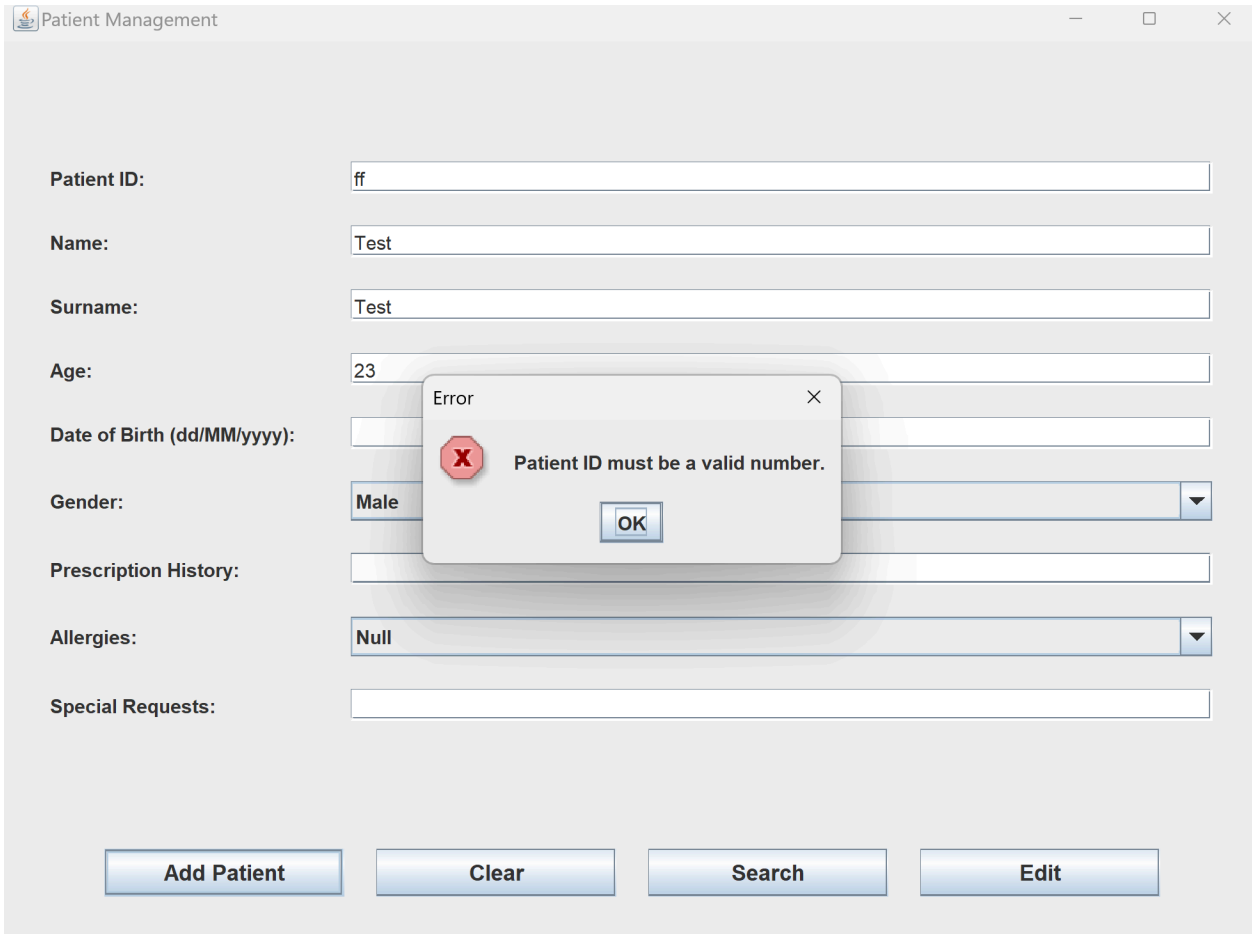
button.setPreferredSize(new Dimension(200, 40));

button.setPreferredSize(new Dimension(**300**, 40));
// Increased width and height

It has also been noted that entering letters instead of positive values for Patient ID and Age does not prompt user that these are not correct:



A validation method has been implemented so that fields Age and Patient ID only accept positive numbers. This change will prevent errors and maintain the logical consistency of the data, as negative values for age or Patient ID are invalid. Additionally, displaying error messages for incorrect inputs will guide users to enter valid data, improving the system's reliability and reducing the risk of processing incorrect information:



Conclusion

Reflecting on the development and implementation of this project, with this being my very first full project, it has been an invaluable learning experience that offered insights into technical development and project management. It is safe to say that it has clearly highlighted the importance of thorough planning and execution, as well as the ability to stay flexible and adapt as challenges arise. While many objectives were achieved successfully, there were also gaps and setbacks that have highlighted areas for improvement and growth in future projects.

One of the key lessons learned was the importance of proper time management and prioritisation. While the MoSCoW prioritisation method helped to ensure that core features were developed, insufficient time allocated to the design phase resulted in some omissions, such as the lack of room allocation features. This once again emphasises the need to find a better balance between prioritising essential features yet still allocating enough time for secondary ones which could provide a better user experience.

Looking for documentation and resources was relatively easy due to the ample online sources available, and the more you research, the more you realise you have yet to learn. It was very easy to get lost in the research aspect and lose track of time.

Finally, the project highlighted the importance of communication and documentation. Gaps in UML diagrams and inconsistencies between documentation and actual implementation pointed to the need for better alignment and regular updates to the project. Peer reviews proved invaluable in identifying these gaps and ensuring that the project met its goals, demonstrating the importance of collaboration and constructive feedback.

Creating this project was not only fun (once you realise the different options there are, the possibilities become endless) but it has also deepened my understanding of time management, effective communication, and adaptability; essential skills for future success in this field. Overall, even though not all methods and functions have made it to the final application, I am quite happy with the outcome. I believe the simple to the point UI creates a user-friendly application, while the code can be very easily updated and improved further depending on future hospital needs, which is ultimately every developer's goal.

References

- AWS. (n.d.). Amazon Web Services (AWS) Cloud Computing Services. Available at: https://aws.amazon.com [Accessed 17 Nov. 2024].
- Beck, K., Beedle, M., van Bennekum, A., et al. (2001). Manifesto for Agile Software Development. Available at: https://agilemanifesto.org [Accessed 17 Nov. 2024].
- Bolscher, R. & Daneva, M., 2019. Designing software architecture to support continuous delivery and DevOps: A systematic literature review. In: M. van Sinderen, L. Maciaszek and L. Maciaszek, eds. *ICSOFT 2019 – Proceedings of the 14th International Conference on Software Technologies*. SCITEPRESS, pp. 27-39. 14th International Conference on Software Technologies, ICSOFT 2019, Prague, Czech Republic, 26 July 2019. Available at: https://www.scitepress.org/Link.aspx?doi=10.5220/0007837000270039 [Accessed 10 November 2024].
- Docker. (n.d.). Docker Documentation. Available at: https://docs.docker.com [Accessed 17 Nov. 2024].
- Figma. (n.d.). Figma: The Collaborative Interface Design Tool. Available at: https://www.figma.com [Accessed 17 Nov. 2024].
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J., 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. 1st ed. Boston: Addison-Wesley.
- Jabbari, R., bin Ali, N., Petersen, K. and Tanveer, B., 2016. What is DevOps? A systematic mapping study on definitions and practices. In: *Proceedings of the Scientific Workshop Proceedings of XP2016*, pp. 1–11. Available at: https://dl.acm.org/doi/10.1145/2962695.2962707 [Accessed 10 November 2024].
- PostgreSQL. (n.d.). PostgreSQL: The World's Most Advanced Open Source Relational Database. Available at: https://www.postgresql.org [Accessed 17 Nov. 2024].
- Pressman, R.S. and Maxim, B.R. (2020). Software Engineering: A Practitioner's Approach. 9th ed. New York: McGraw-Hill Education.
- React.js. (n.d.). React Documentation. Available at: https://reactjs.org [Accessed 17 Nov. 2024].
- Sommerville, I. (2016). Software Engineering. 10th ed. Boston: Pearson.
- Lucidchart, n.d. draw.io Diagramming Tool. [online] Available at: https://www.diagrams.net [Accessed 28 Nov. 2024].

Appendices

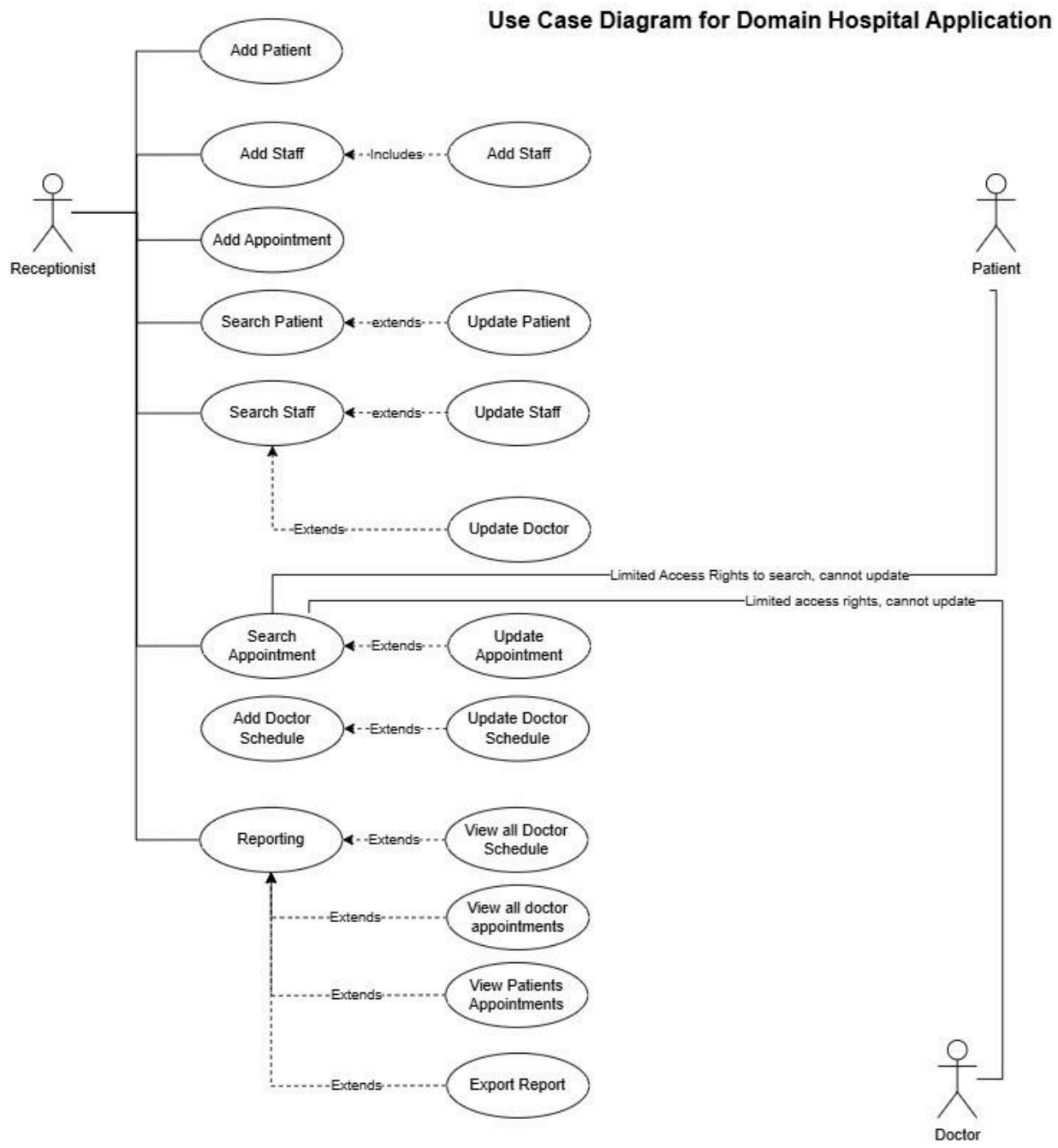


Figure 1: Use Case Diagram showing the user interactions with the system

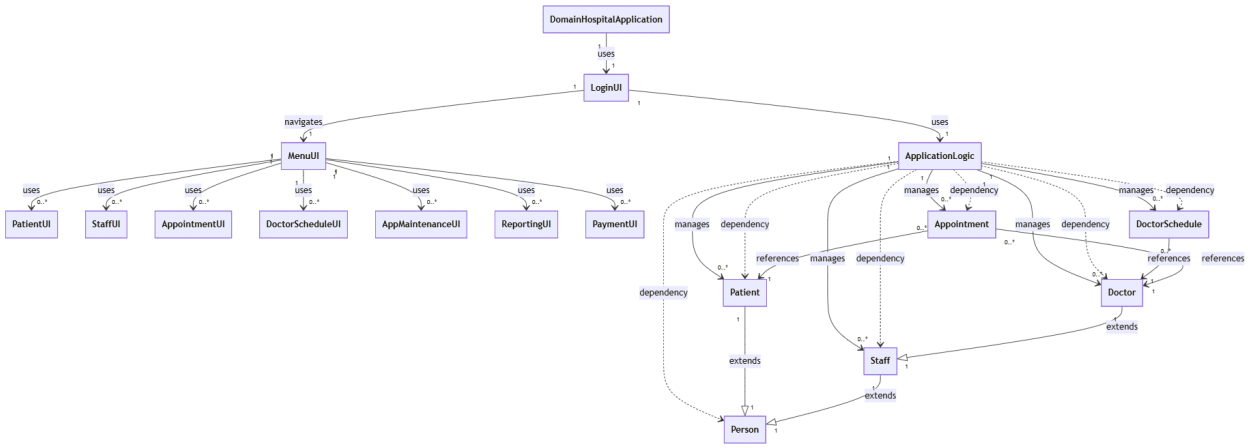


Figure 2: UML Class Diagram depicting the system's classes and their relationships

Figure 3: UML Class Diagram showing the static structure of the system by depicting the system's classes, their attributes and methods

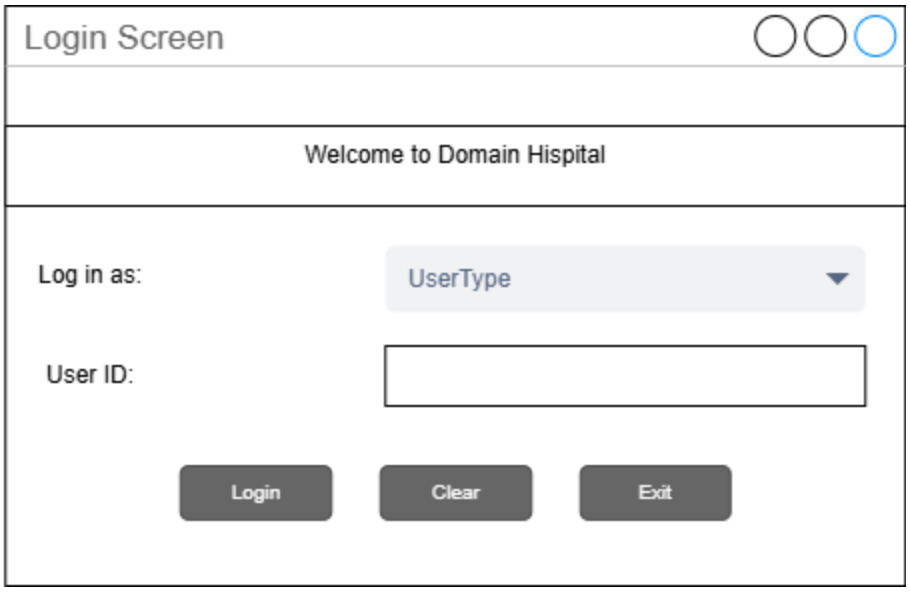


Figure 4: Visual prototype of Login UI

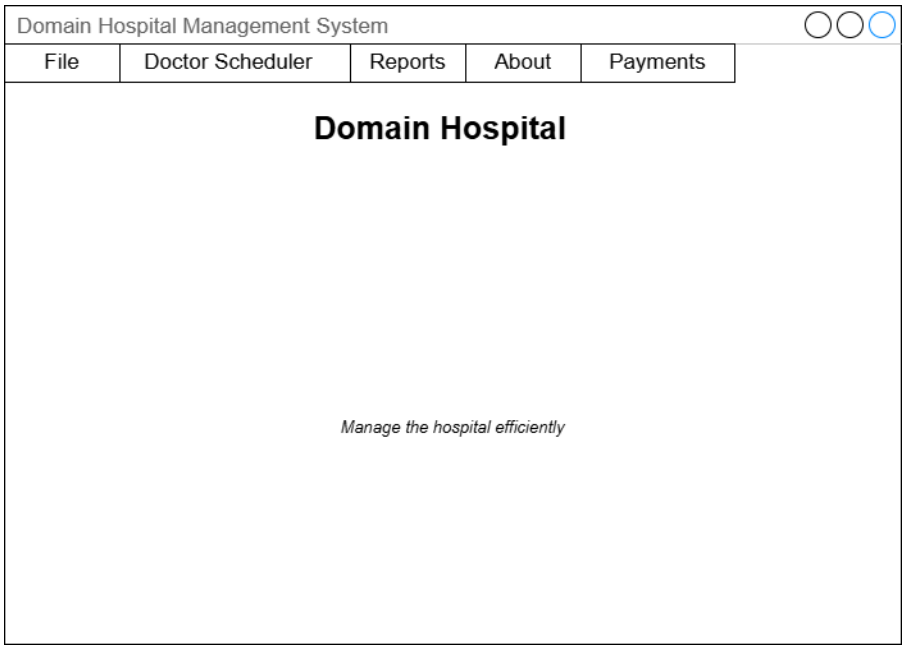


Figure 5 : Visual prototype of Main Menu UI

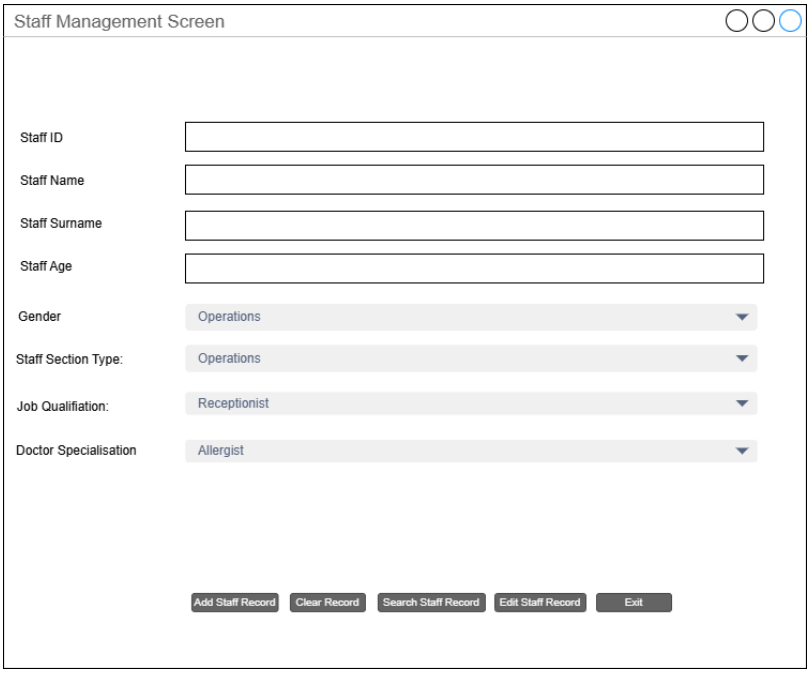


Figure 6 : Visual prototype of Staff Management UI

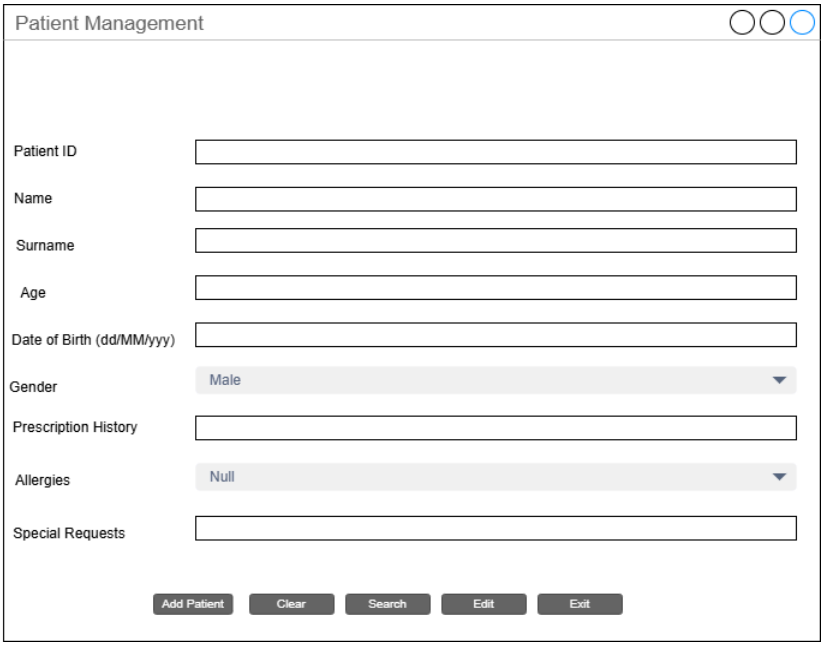


Figure 7 : Visual prototype of Patient Management UI

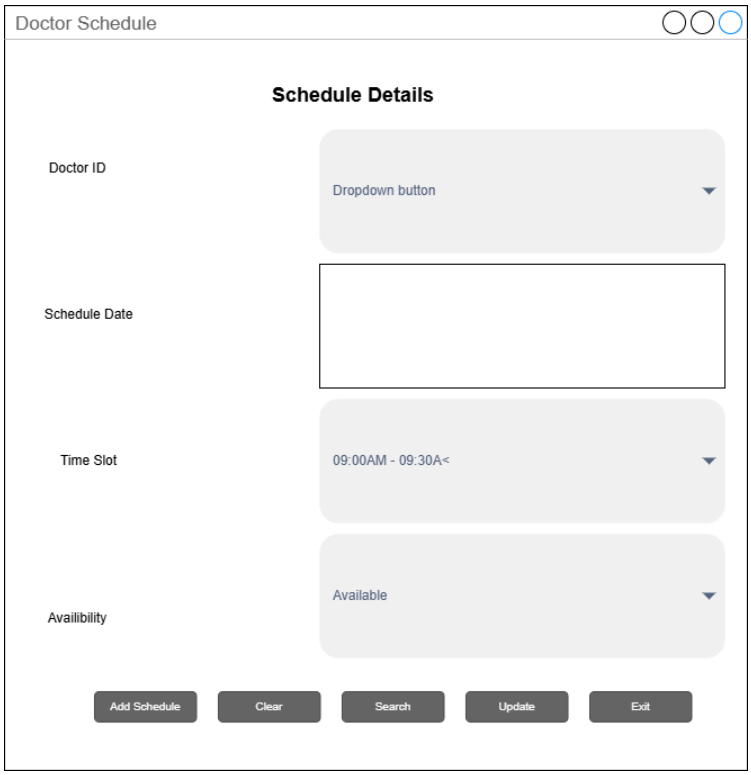


Figure 8 : Visual prototype of Doctor Schedule UI

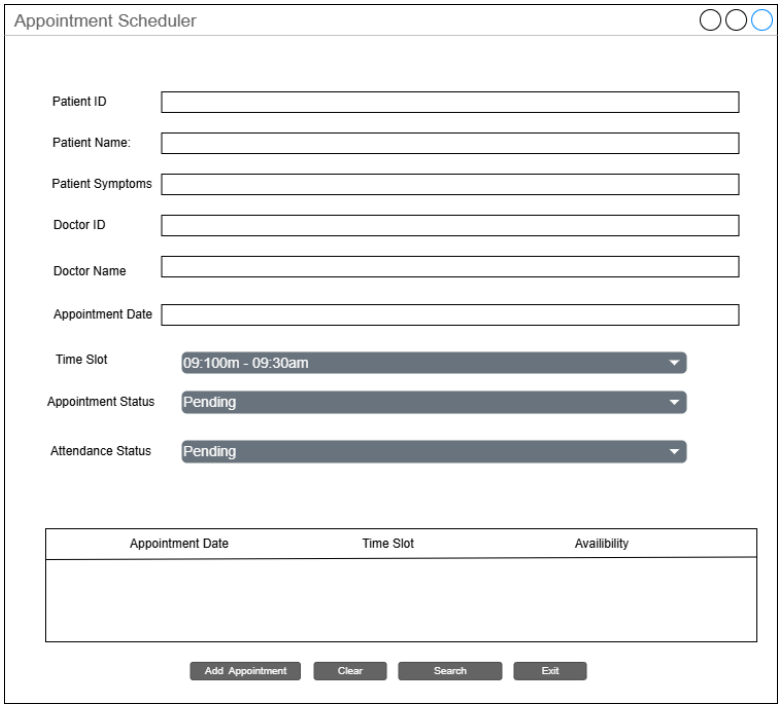


Figure 9 : Visual prototype of Appointment Schedule UI

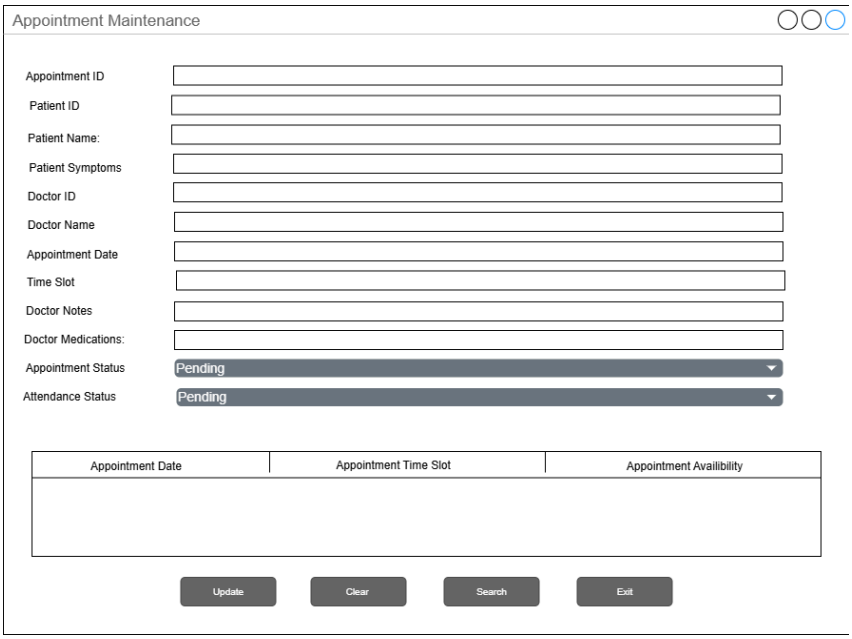


Figure 10 : Visual prototype of Appointment Maintenance UI

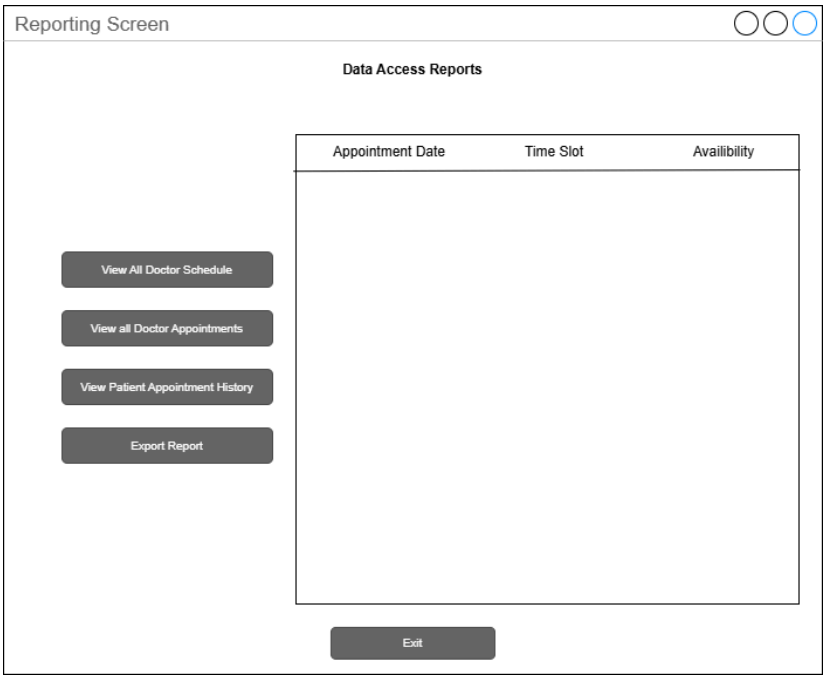


Figure 11 : Visual prototype of Reports UI

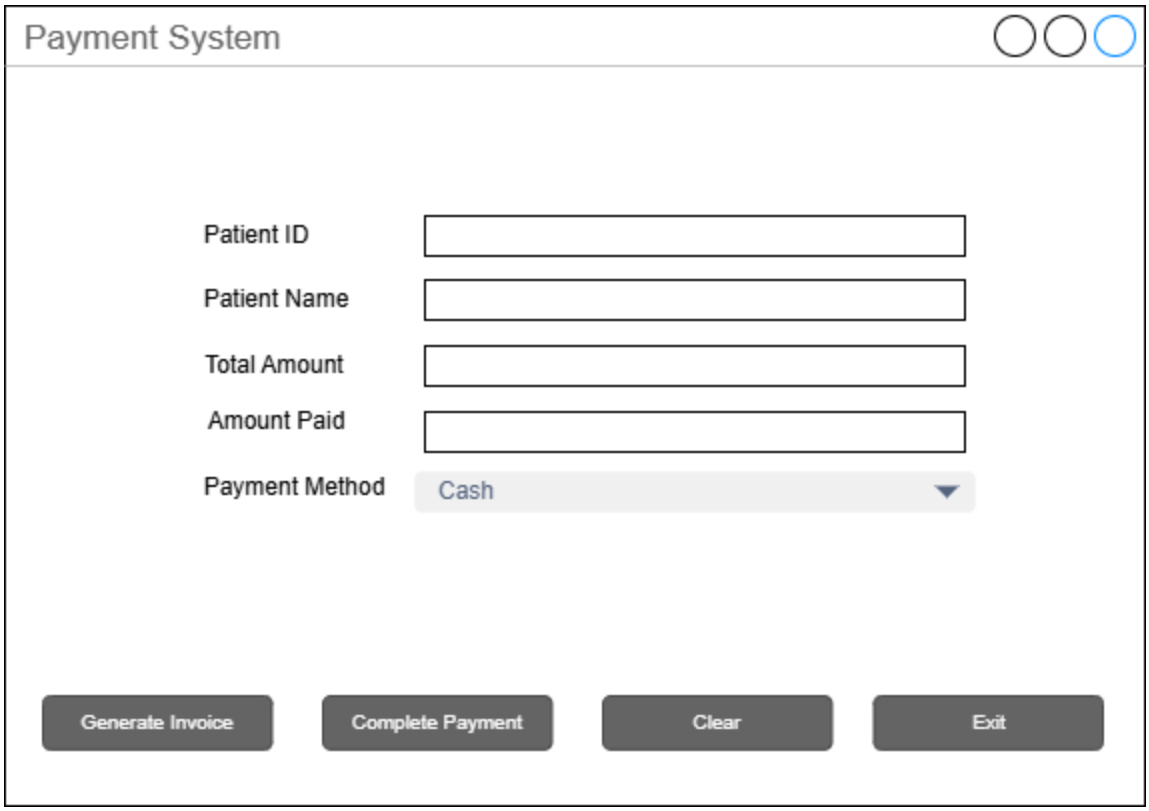


Figure 12 : Visual prototype of Payments UI

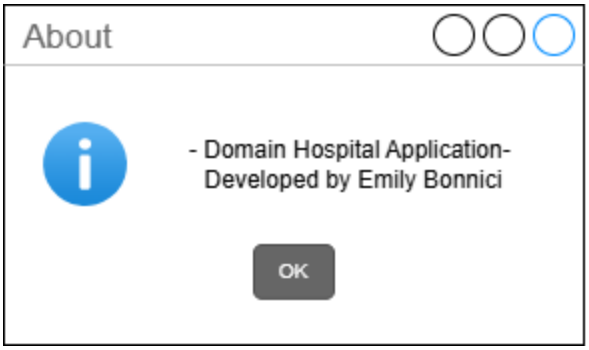


Figure 13 : Visual prototype of About UI